KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

클래스 2 __str__과 __repr__

print했을 때
붕어빵이 보이게 만들기

```
class Bread:
    def __str__(self):
        return "팥 붕어빵 1,000원"

print(a)
# 팥 붕어빵 1,000원
# (전에는 <__main__.Bread object at ...>)
```

오늘의 순서

1	왜 필요한가	print했을 때 낯선 글자, 파이썬이 찾아 부르는 메서드, 가장 짧은 __str__	4–6쪽
2	`__str__`	붕어빵마다 다른 글자, 불리는 곳, intro와 비교	7–9쪽
3	`__repr__`	남는 낯선 글자, '팥'과 팥, 주문표 모양, 한 장 표, 눌러 보기, 하나만 만든다면	10–16쪽
4	붕어빵 여러 개 다루기	for, 컴프리헨션, key 정렬, 재고 바꾸기, if로 막기	17–21쪽
5	오류와 함정	print를 쓴 __str__, 숫자 돌려주기, 밑줄 하나, 글자 더하기	22–25쪽
6	실습	노점의 하루 두 장, 카페 메뉴판	26–28쪽

3쪽은 노트북 준비, 29–30쪽은 막힐 때와 한 장 요약이에요.

들어가기

24에서 이어서 해요

이번에 다시 쓰는 것	챕터
`class, __init__, self`	24
메서드와 `a.intro()`	24
f-string과 칸 맞추기	13
return과 print의 차이	19
`sorted`와 `key=lambda`	22

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 왜 필요한가

숫자와 리스트는 잘 보이는데, 붕어빵은 안 보여요

[1]
```python
print(3)
print("팥")
print([1, 2])
```

```
3
팥
[1, 2]
```

← 넣은 그대로 보여요

[2]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

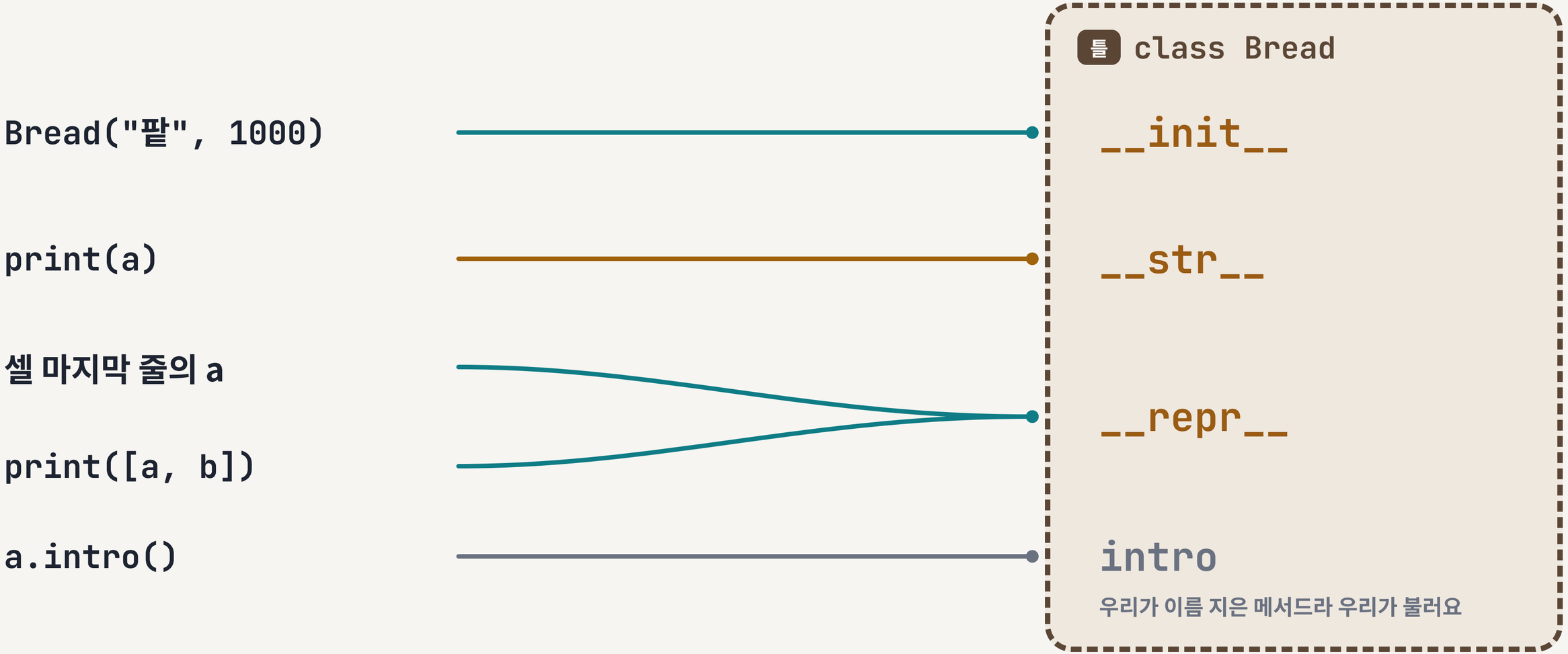
a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
print([a, b])
```

```
<__main__.Bread object at 0x7f6735bb38c0>
[<__main__.Bread object at 0x7f6735bb38c0>, <__main__.Bread object at 0x7f6735dc4a50>]
```

← Bread는 우리가 만든 틀이라 파이썬이 몰라요

1. 왜 필요한가

정해진 순간에 파이썬이 틀에서 찾아 부르는 메서드



앞뒤 밑줄 두 개는 파이썬과 약속한 이름이에요. 틀에 없으면 기본 모양을 써요.

가장 짧은 __str__: 글자 하나를 돌려주기

[3]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return "붕어빵이에요"

a = Bread("팥", 1000)
print(a)
```

← print하지 않고 return해요

붕어빵이에요 ← __str__이 돌려준 글자를 print가 찍어요

2. __str__

__str__에 self의 값을 넣어 붕어빵마다 다르게

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
print(b)
```

← 손님에게 보여 줄 메뉴판 글씨

```
팥 붕어빵 1,000원
슈크림 붕어빵 1,200원
```

← print(a): self가 a

← print(b): self가 b

2. __str__

print 말고도 글자가 필요한 곳에서 불려요

[2]
```python
print(str(a))
print(f"주문: {a}")
print("오늘의 빵:", b)
```

← 12의 타입 바꾸기 str()에 넣어도

← f-string 중괄호 안에 넣어도

← print에 쉼표로 넘겨도

```
팥 붕어빵 1,000원
주문: 팥 붕어빵 1,000원
오늘의 빵: 슈크림 붕어빵 1,200원
```

[3]
```python
msg = "포장할게요: " + str(a)
print(msg)
```

← 바꾼 글자는 더하기로 이을 수 있어요

```
포장할게요: 팥 붕어빵 1,000원
```

파이썬이 붕어빵을 글자로 바꿔야 할 때마다 __str__을 불러요.

2. __str__

intro는 우리가 부르고, __str__은 파이썬이 불러요

24의 intro

```python
    def intro(self):
        print(self.flavor, "붕어빵")

a.intro()
print(a)   # 여전히 <... object>
```

Python

__str__

```python
    def __str__(self):
        return f"{self.flavor} 붕어빵"

print(a)
f"주문: {a}"
```

Python

	intro	__str__
누가 부르나요	우리가 a.intro()로	print, str, f-string이 알아서
무엇을 하나요	안에서 바로 print	글자를 return
다른 글자와 잇기	안 돼요	f-string, 더하기로 이어요

3. __repr__

__str__을 만들었는데도 낯선 글자가 남아요

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a   ← print 없이 셀 마지막 줄에 이름만
```

Python

```
<__main__.Bread at 0x7febaa53c6e0>
```

← __str__을 쓰지 않아요

[2]
```python
print([a, b])   ← 리스트 안의 붕어빵도 __str__을 쓰지 않아요
```

Python

```
[<__main__.Bread object at 0x7febaa53c6e0>, <__main__.Bread object at 0x7febaa7fb890>]
```

3. __repr__

이 차이는 "팥"에서 이미 봤어요

[3]
```
"팥"
```
Python

```
'팥'
```

[4]
```
print("팥")
print(["팥", "슈크림"])
print(str("팥"))
print(repr("팥"))
```
Python

```
팥
['팥', '슈크림']
팥
'팥'
```

str 손님용

팥

print, str()

repr 만든 사람용



셀 마지막 줄, 리스트 안, repr()

글자라는 게 보이게 따옴표까지 붙이는 쪽이 repr이에요.

3. __repr__

__repr__까지 만들면 어디서나 보여요

[5]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

    def __repr__(self):
        return f"Bread('{self.flavor}', {self.price})"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a
```

← 만든 사람이 보는 주문표 글씨
다시 찍는 코드 모양으로 적어요

```
Bread('팥', 1000)
```

← 셀 마지막 줄도 이제 보여요

[6]
```python
print(a)
print([a, b])
```

```
팥 붕어빵 1,000원
[Bread('팥', 1000), Bread('슈크림', 1200)]
```

← print는 여전히 __str__
← 리스트 안은 __repr__

3. __repr__

print는 __str__, 셀과 리스트 안은 __repr__을 찾아요

이렇게 쓰면	불리는 메서드	둘 다 있을 때	아무것도 없을 때
`Bread("팥", 1000)`	`__init__`	찍기	재료 없이 찍기
`print(a), str(a), f"{a}"`	`__str__`	팥 붕어빵 1,000원	`<... object at 0x...>`
셀 마지막 줄의 a	`__repr__`	`Bread('팥', 1000)`	`<... at 0x...>`
`print([a, b])`	안의 값은 __repr__	`[Bread('팥', 1000), ...]`	`[<... object ...>, ...]`
`a.intro()`	우리가 부른 것	우리가 부를 때만	`AttributeError`

손님에게 보여 줄 땐 __str__, 무엇이 들었는지 확인할 땐 __repr__이에요.

3. __repr__

눌러 보기: 지금 누가 불릴까요

```
class Bread:
    def __init__(self, flavor, price): ...
    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"
    def __repr__(self):
        return f"Bread('{self.flavor}', {self.price})"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
```

틀에 **__str__ 있음** **__repr__ 있음** 눌러서 빼기, 다시 넣기

① 이렇게 보여 달라고 하면

`print(a)`	`f"주문: {a}"`
셀 마지막 줄 `a`	`print([a, b])`

② 파이썬이 틀에서 찾는 순서

`__str__` → `__repr__` → 기본 모양

③ 출력

---

①의 버튼을 눌러 보세요. 브라우저로 슬라이드를 열면 눌러져요.

3. __repr__

하나만 만든다면 __repr__이에요

[7]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __repr__(self):
        return f"Bread('{self.flavor}', {self.price})"

a = Bread("팥", 1000)
b = Bread("슈크림", 1000)
print(a)
print([a, b])
```

← __str__ 없이 __repr__만

```
Bread('팥', 1000)
[Bread('팥', 1000), Bread('슈크림', 1200)]
```

← print도 __repr__을 빌려 써요

영수증, 메뉴판처럼 손님용 글씨가 따로 필요할 때 __str__을 더해요.

3. __repr__ · 참고, 읽기만

len과 +도 같은 원리예요

[8]
```python
print(len([1, 2, 3]))
print([1, 2, 3].__len__())
print(3 + 4)
print((3).__add__(4))
```

```
3
3
7
7
```

← len은 리스트 틀의 __len__을 불러요

← +는 숫자 틀의 __add__를 불러요

따라 치지 않아도 돼요. 이 과정에서 직접 만드는 건 __init__, __str__, __repr__ 세 개예요.

4. 붕어빵 여러 개 다루기

붕어빵 여러 개를 리스트에 담아 하나씩 찍어요

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

    def __repr__(self):
        return f"Bread('{self.flavor}', {self.price})"

breads = [Bread("팥", 1000), Bread("슈크림", 1200), Bread("피자", 1500)]
for b in breads:   ← 하나씩 꺼내 print
    print(b)
```
Python

```
팥 붕어빵 1,000원
슈크림 붕어빵 1,200원      ← b는 붕어빵 하나라서 __str__을 써요
피자 붕어빵 1,500원
```

4. 붕어빵 여러 개 다루기

23의 컴프리헨션으로 맛만, 가격만, 비싼 것만

[2]
```python
print([b.flavor for b in breads])
print(sum([b.price for b in breads]))
print([b for b in breads if b.price >= 1200])
```

← 맛 칸만 모아요

← 가격만 모아 sum에 넘겨요

← 비싼 붕어빵을 통째로

```
['팥', '슈크림', '피자']
3700
[Bread('슈크림', 1200), Bread('피자', 1500)]
```

← 리스트 안이라 __repr__

__repr__이 없었다면 마지막 줄은 <... object at 0x...>뿐이었어요.

4. 붕어빵 여러 개 다루기

22의 key=lambda로 가격순 정렬과 가장 비싼 붕어빵

[3]
```python
print(sorted(breads, key=lambda b: b.price, reverse=True))
print(max(breads, key=lambda b: b.price))
```

```
[Bread('피자', 1500), Bread('슈크림', 1200), Bread('팥', 1000)]
피자 붕어빵 1,500원
```

← max는 붕어빵 하나라서 __str__

[4]
```python
print(sorted(breads))
```

← 붕어빵끼리는 무엇으로 비교할지 몰라요

```
TypeError: '<' not supported between instances of 'Bread' and 'Bread'
```

× ~~sorted(breads)~~ → ✓ `sorted(breads, key=lambda b: b.price)`

4. 붕어빵 여러 개 다루기

메서드로 재고와 판매 개수를 바꿔요

[5]
```python
class Stall:
    def __init__(self, name, stock):
        self.name = name
        self.stock = stock
        self.sold = 0

    def sell(self, qty):
        self.stock = self.stock - qty
        self.sold = self.sold + qty

    def __str__(self):
        return f"{self.name}: 남은 {self.stock}개, 판매 {self.sold}개"

s = Stall("팥", 10)
print(s)
s.sell(3)
print(s)
```

← 값 바꾸기

Python

```
팥: 남은 10개, 판매 0개
팥: 남은 7개, 판매 3개
```

일	stock	sold
찍기	10	0
sell(3)	7	3

바뀐 값은 다음에 부를 때까지 남아 있어요. 이걸 상태라고 불러요.

4. 붕어빵 여러 개 다루기

재고가 모자라면 팔지 않게 if로 막아요

[6]
```python
class Stall:
    def __init__(self, name, stock):
        self.name = name
        self.stock = stock
        self.sold = 0

    def sell(self, qty):
        if qty > self.stock:
            return "재고가 모자라요"      ← 17의 if로 막기
        self.stock = self.stock - qty
        self.sold = self.sold + qty
        return "판매 완료"

s = Stall("팥", 5)
print(s.sell(3))
print(s.sell(4))
print(s.stock, s.sold)
```
Python

```
판매 완료                ← 5개 중 3개를 팔았어요
재고가 모자라요          ← 4개는 막혔어요. return에서 바로 끝나요
2 3                      ← 남은 2개, 판매 3개 그대로
```

__str__ 안에서 print하면 returned non-string

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        print(self.flavor, "붕어빵")    ← return이 없어요

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
```

팥 붕어빵
TypeError: __str__ returned non-string (type NoneType) ← 글자 대신 None

× ~~print(self.flavor, "붕어빵")~~ → ✓ `return f"{self.flavor} 붕어빵"`

5. 오류와 함정

숫자를 돌려줘도 returned non-string

[2]
```
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return self.price   ← 가격 숫자를 그대로

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
```
Python

```
TypeError: __str__ returned non-string (type int)   ← int = 숫자
```

× ~~return self.price~~ → ✓ `return f"{self.price}원"`

5. 오류와 함정

밑줄이 하나면 오류 없이 그냥 무시돼요

[3]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def _str_(self):   ← 밑줄이 하나씩이에요
        return f"{self.flavor} 붕어빵"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a)
```

Python

```
<__main__.Bread object at 0x7fc8a2a6bcb0>   ← 오류 없이 기본 모양
```

✕ ~~def _str_(self):~~ → ✓ `def __str__(self):`

5. 오류와 함정

글자에 붕어빵을 바로 더하면 can only concatenate

[4]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print("주문: " + a)   ← 더하기는 __str__을 부르지 않아요
```

Python

```
TypeError: can only concatenate str (not "Bread") to str
```

× ~~"주문: " + a~~ → ✓ `"주문: " + str(a)`

f"주문: {a}"나 print("주문:", a)처럼 적어도 __str__이 불려요.

6. 실습

붕어빵 노점의 하루 ①: 노점 틀 만들기

[1]

```python
class Stall:
    def __init__(self, flavor, price, stock):
        self.flavor = flavor
        self.price = price
        self.stock = stock
        self.sold = 0

    def sell(self, qty):
        if qty > self.stock:
            return f"{self.flavor} 재고가 모자라요"
        self.stock = self.stock - qty
        self.sold = self.sold + qty
        return f"{self.flavor} {qty}개 판매"

    def __str__(self):
        return f"{self.flavor}: 판매 {self.sold}개, 매출 {self.sold * self.price:,}원"
```

Python

__init__: 재료 셋, 판매는 0

sell: if로 막고 상태 바꾸기

__str__: 매출 글씨

출력이 없는 셀이에요. 틀을 만들기만 했어요.

6. 실습

붕어빵 노점의 하루 ②: 팔고, 막고, 매출 찍기

[2]
```python
pat = Stall("팥", 1000, 20)
cream = Stall("슈크림", 1200, 10)
print(pat.sell(5))
print(cream.sell(12))
print(cream.sell(4))
print(pat)
print(cream)
```

```
팥 5개 판매
슈크림 재고가 모자라요
슈크림 4개 판매
팥: 판매 5개, 매출 5,000원
슈크림: 판매 4개, 매출 4,800원
```

일	팥 stock	슈크림 stock
찍기	20	10
pat.sell(5)	15	10
cream.sell(12)	15	10
cream.sell(4)	15	6

12개는 재고 10개보다 많아서 if에서 return했어요. 재고는 그대로예요.

__init__으로 찍고, 메서드로 상태를 바꾸고, if로 막고, __str__로 찍었어요.

6. 실습

카페 메뉴판: 손님용 줄 맞춤과 개발자용 확인

[3]

```python
class Drink:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __str__(self):
        return f"{self.name:<7}{self.price:>6,}원"

    def __repr__(self):
        return f"Drink('{self.name}', {self.price})"

menu = [Drink("latte", 5000), Drink("tea", 4000), Drink("coffee", 4500)]
for d in sorted(menu, key=lambda d: d.price):
    print(d)
print([d for d in menu if d.price < 5000])
```

Python

← 손님용: 13의 칸 맞추기

← 개발자용: 코드 모양

```
tea     4,000원
coffee  4,500원
latte   5,000원
[Drink('tea', 4000), Drink('coffee', 4500)]
```

가격순 메뉴판 줄은 __str__

← 골라 담은 리스트는 __repr__

정리

막히면 오류의 마지막 줄을 봐요

보이는 것과 이유	고치기
`<__main__.Bread object at 0x...>` __str__이나 __repr__이 없거나 밑줄이 하나예요	**__repr__ 만들기**
리스트 안만 <... object ...> __str__만 만들었어요	**__repr__도 만들기**
`TypeError: __str__ returned non-string (type NoneType)` 안에서 print했어요	**return으로**
`TypeError: __str__ returned non-string (type int)` 숫자를 돌려줬어요	**str()이나 f-string**
`TypeError: can only concatenate str (not "Bread") to str` 글자에 붕어빵을 바로 더했어요	**`str(a), f"{a}"`**
`TypeError: '<' not supported between instances` 붕어빵끼리 비교할 기준이 없어요	**`key=lambda b: b.price`**
재고가 마이너스가 돼요 메서드에서 값을 확인하지 않았어요	**if와 return으로 막기**

낯선 글자가 나오면 "지금 파이썬이 어느 메서드를 찾았을까"부터 생각해요.

한 장 요약

`def __str__(self):`

print, str(), f-string에서 보일 글자

`def __repr__(self):`

셀 마지막 줄, 리스트 안에서 보일 글자

`return f"..."`

두 메서드 모두 꼭 글자를 돌려줘요

`Bread('팥', 1000)`

repr은 다시 찍는 코드 모양으로

`str(a)`

글자로 바꿔 다른 글자와 잇기

`__repr__`만

하나만 만든다면. print도 빌려 써요

`key=lambda b: b.price`

붕어빵을 가격으로 줄 세우기

`self.stock - qty`

메서드로 상태 바꾸기, if로 막기

__str__은 손님용 메뉴판, __repr__은 만든 사람용 주문표예요.

다음 챕터: 26에서 틀을 물려받아 슈크림 붕어빵 틀을 만들어요.